LangGraph

What is LangGraph?



- A low-level orchestration framework and runtime for building, managing, and deploying long-running, stateful AI agent
- Provides control and infrastructure for complex agent workflows beyond simple loops. It allows you to mix deterministic code with LLM-driven steps reliably
- Position in the LangChain Ecosystem :
  - LangGraph is the orchestration runtime (durable execution, streaming, HITL).
  - LangChain is the higher-level agent framework (abstractions, integrations).
  - LangSmith is the platform for tracing, evaluation, and deployment
- Inspired by Google's Pregel and Apache Beam; public interface draws from NetworkX

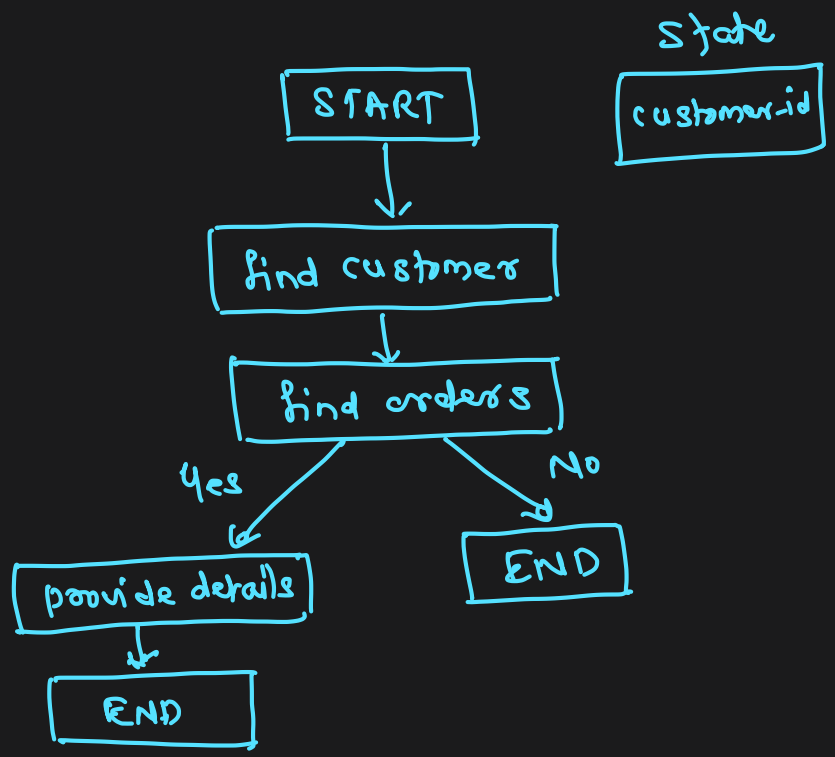
Benefits

- **Durable Execution (Persistence):** Agents can survive failures and be paused/resumed, crucial for long-running tasks
- **Human-in-the-Loop (HITL):** Ability to pause the graph and wait for human input, review, or approval before proceeding (e.g., for sensitive actions like sending an email)
- **Comprehensive Memory:**
  - **Short-Term Memory (STM):** The state within a single thread, persisted via **Checkpointers** (keyed by thread_id). Used for conversation continuity and resuming after interruptions
  - **Long-Term Memory (LTM):** Information persisted across threads/sessions (keyed by user_id/namespace). Stored via the **Store** API for user preferences or facts
- **Streaming:** Allows you to see agent steps and tokens in real-time as they are generated
- **Debugging & Observability with LangSmith:** Provides full trace trees, latency breakdowns, and token/cost tracking to debug complex behavior

Core Architecture - State

- The **shared**, **persistent data** that flows through the graph. It acts as the agent's working memory
- Typically defined as a TypedDict
- LangGraph maintains a shared **state** as the agent executes
- Nodes can read and update this state.

```
state = {
    "user_message": "Where is my order?",
    "order_id": "ORD123",
    "order_status": None,
    "needs_refund": False,
    "human_approved": False
}
```



Core Architecture – Node

- **Units of work**
- Each node is a function (e.g., calling an LLM, using a tool, executing code)
- Nodes receive the current state, perform an action, and return an update to the state

```python
def find_order(state):
    order = get_order_from_database(state["order_id"])

    return {
        "order_status": order["status"]
    }
```

updated state

```python
def check_refund(state):
    if state["order_status"] == "cancelled":
        return {"needs_refund": True}

    return {"needs_refund": False}
```

updated state

Core Architecture - Edges



- Define the **flow of control** between nodes
- **Normal Edges**
  - Directly connect one node to the next in a deterministic path (`graph.add_edge("node_a", "node_b")`)
- **Conditional Edges** → branching
  - Use a routing function to choose the next node(s) based on the current state (`graph.add_conditional_edges("node_a", routing_function)`)
- **Entry Point**
  - The first node to execute, defined from the special START node

Core Architecture - Loop (workflow)



- This is one of the biggest differences between a simple chain and an agentic workflow
- The agent can **loop until it reaches a suitable stopping condition**
- This is very useful for implementing patterns such as **ReAct**, where the agent repeatedly reasons, acts, observes, and decides what to do next

Key Concepts



- **Checkpointers** (MemorySaver, PostgresSaver, SqliteSaver)
  - The mechanism for saving graph state
  - MemorySaver is for development; use PostgresSaver (or SqliteSaver) for production to persist state across restarts
  - **Only the parent graph should be compiled with a checkpointer when using subgraphs** to avoid state duplication and bloat
- Interrupt
  - The primitive used to implement HITL. When a node calls interrupt(), execution pauses, and the graph returns the payload for the human to review. Resuming is done by passing a Command(resume=...)
- Command
  - Versatile primitive for controlling graph execution.
  - It allows a node to both update state and navigate to specific nodes (goto) in a single step
- Send
  - Used for advanced map-reduce patterns where the number of edges is unknown at compile time
  - For example, a node might generate a list of subjects and Send each one to a "generate_joke" node

When to use?

- Multi-step workflows
- Conditional branching
- Loops and iterative workflows
- Human-in-the-loop
- Stateful applications
- **Retry and error handling**
- Multiple agents
- Long-running workflows

When not to use?



- Simple LLM calls
- Simple chains
- Simple RAG
- Simple tool calling
- When the workflow doesn't require state
- Very small applications

Scenarios



- Customer Support Agent
- Research Agent
- Plan-and-Execute Agent
- Coding Agent
- Document Processing
- Financial/Business Approval Workflow
- Medical/Legal Document Workflow
- Multi-Agent Research System
- Customer Onboarding

Famous companies using LangGraph

Company	Application	What LangGraph is doing
LinkedIn	AI Recruiter	Hierarchical agents for candidate search, matching and recruitment workflows
Uber	Developer Platform	Agents for code migration and automated unit-test generation
Klarna	AI Customer Assistant	Multi-step customer-support and payment workflows
Replit	Replit Agent	Coding agents that build software, with human-in-the-loop capabilities
Elastic	AI / Security	Agentic workflows for security and other complex tasks
J.P. Morgan	AI Copilot	Domain-specific enterprise copilot
Rakuten	AI Copilot	Domain-specific agent workflows
Morningstar	AI Research Assistant	Research and summarization
Vodafone	AI applications	Code generation and internal search
Qodo	Coding AI	Code-generation workflows

Langchain vs LangGraph

- **Choose LangChain when**
  - You need a simple LLM application
  - You are building basic RAG
  - You need prompt templates
  - You need model/tool integrations
  - You need a straightforward agent
  - Your workflow is mostly linear

- **Choose LangGraph when**
  - Your workflow has **multiple steps**
  - You need **shared state**
  - You have **conditional branching**
  - You need **loops**
  - You need **retry/re-evaluation**
  - You need **human approval**
  - You have **multiple agents**
  - You need **pause/resume**
  - You need more control over agent execution